Study guide v3: finding communities in hypergraphs

Read sections 1 and 2 out loud twice. Then skim section 7 (questions) once. Everything else is for reference.

---

1. The one-minute version

Say this without notes.

1. Real groups are hyperedges. A committee is one hyperedge with many nodes, not a set of pairs.
2. The usual shortcut turns every hyperedge into pairs. That forgets the pairs came from one hyperedge, and it makes big hyperedges count far too much.
3. Kovacs, Benedek and Palla (2025) find communities by chaining hyperedges that overlap enough. They give two versions of "enough".
4. Their method has four settings, and the paper leaves them to the user. It tests only an easy case, where intra-community hyperedges and random hyperedges have different sizes.
5. We rebuilt the method and checked it in the paper's setting. It recovers the ground-truth partition exactly when the right version is used.
6. We added a rule that picks the settings without any labels.
7. The rule scores 0.555 and 0.586 for the two versions. The paper's default scores 0.292 and 0.229. The oracle scores 0.644 and 0.698.
8. So the rule beats the default by a wide margin and still trails the oracle.
9. On real committee data, the rule gives no gain. We report that as a limit.

Vocabulary

- **Node**: one element of the network. Examples here are a, b, c, d.
- **Hyperedge**: a set of any number of nodes that belong together.
- **Size**: the number of nodes in a hyperedge.
- **Hypergraph**: a set of nodes plus a set of hyperedges.
- **Community**: a set of nodes that share many hyperedges with each other and few with other nodes.
- **Partition**: an assignment where every node is in exactly one community.
- **Cover**: an assignment where communities may overlap and some nodes may be in none.
- **Ground-truth partition**: the planted communities we generate.
- **Detected cover**: the communities the algorithm outputs.
- **Intra-community hyperedge**: a hyperedge whose nodes all come from one planted community.

- **Random hyperedge**: a hyperedge whose nodes are sampled uniformly from all nodes, so it usually crosses communities.
- **Noise fraction (p)**: the number of random hyperedges per size divided by N.
- **ECS**: a score from 0 to 1 that compares the ground-truth partition with the detected cover.
- **Coverage**: the fraction of nodes placed in at least one detected community.
- **Oracle**: the settings chosen by looking at the ground truth. An upper bound.
- **Default**: the settings the paper suggests with no tuning.
- **Rule**: our label-free choice of settings.

---

2. The story in five beats

Beat 1. Groups are hyperedges. Nodes a, b, c and d share one hyperedge. That is one hyperedge of size 4. Turning it into six pairs loses the fact that it was one hyperedge.

Beat 2. The paper chains overlapping hyperedges. Start from one hyperedge. Add any hyperedge that shares enough nodes with the current community. Repeat until no hyperedge can be added. The union of nodes is one community. Two versions of "enough": one keeps only large hyperedges (large-edge variant), one keeps only small hyperedges (small-edge variant).

Beat 3. The paper leaves a gap. The four settings are left to the user, and the test is easy: size alone tells intra-community hyperedges from random hyperedges. A real user has no ground truth and no such luck.

Beat 4. We rebuilt it and checked it (E1). In the paper's own setting the matching version scores a perfect 1.000 at every noise level. The wrong version falls apart. This tells you our code does what the paper says.

Beat 5. We added a rule for the settings (E3). It needs no labels. It beats the default and trails the oracle. This is our contribution.

Two more studies are backup. E2 asks what happens when the size cut is not clean. E4 tries real committee data.

---

3. Concepts in plain words

Hyperedge and size

A hyperedge is a set of nodes that belong together. Its size is the number of nodes in it. Example: {a, b, c, d} is a hyperedge of size 4. {a, b} is a hyperedge of size 2, which is an ordinary edge. Say: "A graph edge joins two nodes. A hyperedge joins any number."

Hypergraph

A hypergraph is a set of nodes and a set of hyperedges over them.

Flattening and what it loses

Flattening replaces each hyperedge by every pair of nodes inside it. The hyperedge {a, b, c, d} becomes six pairs: ab, ac, ad, bc, bd, cd. A hyperedge of size s gives s(s-1)/2 pairs. Two losses. Big hyperedges count too much: size 8 gives 28 pairs, size 3 gives 3. And the pairs forget they came from one hyperedge: six pairs could just as well be six separate edges. Say: "Flattening turns one hyperedge into many pairs and forgets they belong together."

Community

A community is a set of nodes that share many hyperedges with each other and few with other nodes. Example: nodes a, b, c, d share many hyperedges. They share few with e, f, g. So {a, b, c, d} is a community.

Partition versus cover

A partition puts every node in exactly one community. A cover lets communities overlap and lets some nodes be in none. Example with nodes a, b, c, d, e. Partition: {a, b} and {c, d, e}. Cover: {a, b, c} and {c, d}, with e in none. Node c is in two communities. Say: "Percolation gives a cover, so overlap and left-out nodes are possible."

Ground-truth partition and detected cover

The ground-truth partition is the set of planted communities we generate. The detected cover is the set of communities the algorithm outputs. Both are assignments of the same node set. ECS compares them.

Clique percolation in ordinary graphs

Take a graph. A k-clique is k nodes that are all joined to each other. For k = 3 that is a triangle. Two cliques are adjacent if they share k-1 nodes. Roll across adjacent cliques. All nodes touched form one community. Example, k = 3. Triangle a-b-c and triangle b-c-d share two nodes (k-1), so they join. The community is {a, b, c, d}. Triangle d-e-f shares only d, so it stays apart. Nodes in no triangle are left out. Say: "Roll a clique across adjacent cliques that share all but one node."

The paper's algorithm (five steps)

1. Start from one hyperedge.
2. Add any other hyperedge that shares enough nodes with the current community.
3. Repeat until no hyperedge can be added.
4. The union of nodes is one community.
5. Start again from a hyperedge not yet used.

The communities come from this rule, nothing else. The paper does not guess or train anything. The only open question is what "enough" means. That is where the settings come in.

Large-edge variant (settings k and I_abs)

Use only hyperedges of size at least k. Two hyperedges are adjacent if they share at least I_abs nodes. Use it when large hyperedges are the real communities and small ones are chatter. Example: k = 3, I_abs = 2. {a, b, c} and {b, c, d} share two nodes, so they are adjacent. {d, e, f} shares only d, so it stays out. The hyperedge {a, b} is dropped, since its size is below 3. Say: "Throw away small hyperedges, then chain large hyperedges that share enough nodes."

Small-edge variant (settings K and I_rel)

Use only hyperedges of size at most K. A hyperedge of size S joins a community if at least floor(I_rel times S) of its nodes are already in it. Use it when huge hyperedges mean nothing. Example: K = 4, I_rel = 0.5. The current community is {a, b, c, d}. The hyperedge {c, d, e, f} has size 4, so it needs floor(0.5 times 4) = 2 nodes inside. It has c and d, so it joins and e and f enter. The hyperedge {d, x, y, z} has only one node inside, so it stays out. Say: "Throw away huge hyperedges, then let a hyperedge join when half or more of its nodes are already in."

Why size-2 hyperedges are dropped on real data: a pair with one node in the community has floor(0.5 times 2) = 1 node needed, so it always joins. Pairs would glue everything together. The paper drops them, so do we.

The test bed

We generate N = 500 nodes in 10 planted communities of 50. We make two kinds of hyperedges:

- Intra-community hyperedges. All nodes come from one planted community. Example: five nodes, all from community 3.
- Random hyperedges. Nodes are sampled uniformly from all 500, so they almost always cross communities. Their job is to trick the algorithm into merging communities.

The noise fraction p sets how many random hyperedges exist: p times 500 per size. Worked example at p = 0.4: 200 random hyperedges per size, three sizes, so 600 random hyperedges. The intra-community hyperedges are 10 communities times 50 hyperedges times 3 sizes, so 1500. At p = 0.4 about 3 hyperedges in 10 are random.

Say: "Random hyperedges cross communities by design, to confuse the algorithm."

Clean size cut versus partial versus full overlap

"Size cut" means: can you tell intra-community hyperedges from random hyperedges by size alone?

- Clean cut. Intra-community hyperedges have sizes 6 to 8, random hyperedges have sizes 3 to 5 (or the mirror). Keep only hyperedges of size 6 or more and every random hyperedge is gone. This is the paper's test.
- Partial overlap. Intra-community 5 to 7, random 4 to 6. Sizes 5 and 6 appear in both, but size 7 is pure intra-community. One pure size is enough to filter on.
- Full overlap. Both 4 to 6. Size tells you nothing.

"The cut is not clean" means some sizes are shared, so no size filter removes all random hyperedges.

ECS (the score)

ECS compares two assignments of the same node set and gives a number from 0 to 1. Here the two are the ground-truth partition and the detected cover. 1 means identical. Near 0 means unrelated.

How it works, node by node. For node a, list the nodes that share a community with a in the ground truth. Then list the nodes that share a community with a in the detected cover. If the lists agree, node a scores high. Average over all nodes.

Two details. Nodes the algorithm left out count as a community of one. Communities may overlap. That is why we need ECS: plain accuracy cannot handle either.

Toy fact: 100 nodes, the ground truth is two perfect communities of 50, the detected cover puts every node alone. ECS is about 0.02. Almost no node shares a community with the same nodes.

Say: "For every node, ECS asks whether its set of co-members is the same in both assignments."

Coverage

The fraction of nodes placed in at least one detected community. If 450 of 500 nodes are in a community, coverage is 0.9.

Oracle, default and rule

- Oracle: the settings chosen by looking at the ground truth. A cheat, so an upper bound.
- Default: the settings the paper suggests with no tuning.
- Rule: our pick, made without labels.

---

4. The two experiments we present

E1: does our code do what the paper says? (30 runs per cell)

How to read the heat map. Each small square is one pair of settings. Rows are the intersection setting (I_abs or I_rel). Columns are k or K. Dark means good ECS. The top row is noise 0 and the bottom row is noise 0.4. A good setting shows as a dark patch that survives down to the bottom.

Large-edge communities (intra-community hyperedges of size 6 to 8, random hyperedges of size 3 to 5):

noise	large-edge variant	small-edge variant
0	1.000	1.000
0.1	1.000	0.711

noise	large-edge variant	small-edge variant
0.2	1.000	0.571
0.3	1.000	0.482
0.4	1.000	0.424

Mirror case (intra-community hyperedges of size 3 to 5, random hyperedges of size 6 to 8):

noise	large-edge variant	small-edge variant
0	1.000	1.000
0.1	0.400	1.000
0.2	0.282	1.000
0.3	0.233	1.000
0.4	0.206	1.000

Both versions are exact at noise 0.

One detail the paper also notes: in the large-edge variant, I_abs of 4 or more breaks communities apart (ECS 0.86 to 0.91, even at noise 0). Two hyperedges of size 6 to 8 rarely share 4 nodes, so the chain snaps.

Say: "The right version stays perfect. The wrong version collapses. That is the paper's claim, and our code shows it."

E3: a rule for the settings, without labels

The rule in five steps.

1. Try every setting on a grid.
2. For each, look at the community sizes it produced.
3. Throw out a setting if it gives fewer than two communities, or if the biggest community is more than twice the second biggest. That means one community swallowed the rest.
4. Score the survivors by concentration: are the communities few and large? (Each node splits its weight equally over its communities. Concentration is the sum of squared community sizes divided by n squared.)
5. Pick the top score. If nothing survives, use the paper's default.

Worked toy. 100 nodes, the ground truth is two communities of 50.

- A loose setting gives one community of 100. Fewer than two communities, so it is rejected.
- A strict setting gives about 40 tiny communities. They pass, but concentration is about 0.03.
- The middle setting gives two communities of 50. Concentration is (50x50 + 50x50) / 100x100 = 0.5. It wins, and it is the true answer.

Why the first version failed. The first version picked the highest coverage. But a bad setting can give many tiny overlapping communities that together touch every node, so coverage reads near 1. Coverage rewards fragments. Concentration rewards a few large communities.

Numbers (1050 hypergraphs per variant, averaged over all layouts and noise levels):

	rule	coverage rule	oracle	default	hits oracle
large-edge	0.555	0.547	0.644	0.292	46.6%
small-edge	0.586	0.544	0.698	0.229	55.4%

Where it works. When the variant matches a layout with one pure hyperedge size, the rule is near the oracle (rule against oracle): 0.936 vs 1.000, 0.983 vs 0.999, 1.000 vs 1.000, 0.974 vs 0.974.

Where it does not.

- Wrong variant for the layout: 0.263 vs 0.548, 0.286 vs 0.493, 0.233 vs 0.351, 0.226 vs 0.399.
- Full overlap: 0.396 vs 0.471 (large-edge) and 0.410 vs 0.476 (small-edge).

Say: "The rule picks settings without labels. It more than doubles the default, and it still trails the oracle."

---

5. Backup experiments

E2: what if the size cut is not clean? Each method is scored at its best settings per cell.

- Clean cut: the matching variant stays at 1.000 up to noise 0.8.
- Partial overlap: intra-community 5-7 vs random 4-6 gives 0.999 for the large-edge variant. Intra-community 4-6 vs random 5-7 gives about 0.97 for the small-edge variant. One pure size is enough.
- Full overlap, 4-6 vs 4-6: collapses to 0.601 at noise 0.1 and about 0.24 at noise 0.8, for both variants.
- Clique percolation on the flattened graph timed out or ran out of memory in 278 of 350 runs. Where it finished under noise, it scored 0.100 to 0.105. Big hyperedges flatten into near-complete graphs, which are hopeless to search for cliques. That is a finding about cost, not about accuracy.

E4: real committee data. House: 1290 nodes, 341 hyperedges. Senate: 282 nodes, 315 hyperedges. We score against party labels as the ground-truth partition.

- Best ECS is about 0.5, from one or two communities covering every node. That is the trivial score for a two-party split.
- On the House with the large-edge variant, no setting passes the giant-community test, so the rule falls back to the default.
- The rule gives no gain here. Say it as a limit. Party is a weak ground truth for committees.

---

6. Limits

- Planted communities are disjoint, so overlapping communities are never tested.

- Two party labels are a weak ground truth.
- The algorithm leaves some nodes out, and ECS punishes that.
- The rule is a heuristic. Its giant-community test assumes communities of similar size.
- On real committee data the rule does not help, and for the small-edge variant it does worse than the default.

---

7. Twelve likely questions

1. Is ECS 1.0 suspicious? No. At noise 0 the planted communities are the only structure. With the right variant and a pure size, random hyperedges are filtered by size, so the score stays at 1.

2. Why not modularity? Modularity is for ordinary graphs and assumes each node is in one community. We want overlap and we want to avoid flattening. We also have the ground truth, so comparing to it is more direct.

3. Why ECS and not NMI or accuracy? Those need every node in exactly one community. Our detected covers overlap and leave nodes out. ECS handles both, and the paper uses it.

4. What is noise fraction 0.4, concretely? 200 random hyperedges per size, 600 in total, against 1500 intra-community hyperedges.

5. Why did the rule not help on real data? The party labels give only a trivial best score of about 0.5 from one big community. The rule hunts for several large communities, which the labels do not reward. On the House with the large-edge variant it fell back to the default.

6. Did you reproduce the paper? No. We re-implemented it and checked it in the paper's setting. We do not claim to match its figures. We use 30 runs, the paper uses 100.

7. Why drop pairs on real data? A pair with one node in the community always joins in the small-edge variant. Pairs would glue everything. The paper drops them.

8. What would you do next? Test overlapping planted communities, use richer labels than two parties, and design a rule for communities of different sizes.

9. How long does it run? The hyperedge-based algorithm finished every run. A full E2 map of 2275 jobs took about 2547 seconds on 40 processes. Clique percolation on the flattened graph often hit the 60 second limit. We did not do a formal complexity study.

10. Why 30 runs, not 100? Time. The spread across runs was tiny (standard error 0.000 to 0.001 in E1), so more runs would not change the picture.

11. What is the oracle? The best settings for each hypergraph, chosen by looking at the ground truth. It shows the ceiling. It is not something a user could do.

12. What does "the cut is not clean" mean? Intra-community hyperedges and random hyperedges share some sizes, so you cannot remove all random hyperedges by filtering on size. If one size is still pure, the algorithm survives. If none is, it fails.

---

8. Slide-by-slide script (new deck, 20 slides)

1. Title. This is a re-implementation of a 2025 paper on finding communities in hypergraphs. We check it, we test where it breaks, and we add a rule for its settings. We do not claim to reproduce the paper's figures.

2. Hyperedges, not pairs. Nodes a, b, c and d in one hyperedge is one object. Flattening it into six pairs forgets that, and makes big hyperedges count too much. Many real groups, like committees, are hyperedges.

3. Communities, and two ways to assign them. A community is a set of nodes that share many hyperedges. A partition gives each node one community. A cover allows overlap and left-out nodes. The algorithm in this paper gives a cover.

4. Clique percolation in ordinary graphs. A clique is a set of nodes that are all joined to each other. Two cliques are adjacent when they share all but one node. Chain adjacent cliques and you get a community.

5. The paper's idea: chain overlapping hyperedges. Skip the pairs. Start from one hyperedge and add any hyperedge that shares enough nodes with the current community. The union of nodes is one community. Nothing is guessed. It is a rule.

6. Two versions of adjacency. The large-edge variant keeps hyperedges of size at least k and calls two of them adjacent if they share at least I_abs nodes. The small-edge variant keeps hyperedges of size at most K and lets a hyperedge of size S join if at least floor(I_rel times S) of its nodes are already in. Use the first when small hyperedges are chatter, the second when huge hyperedges are meaningless.

7. What the paper leaves open. The four settings are left to the user. The test is easy, because size alone separates intra-community hyperedges from random hyperedges. And the comparison to clique percolation is by picture only.

8. The test bed. 500 nodes in 10 planted communities of 50. Intra-community hyperedges come from inside one community. Random hyperedges are sampled from all nodes. At noise 0.4 there are 600 random hyperedges against 1500 intra-community hyperedges.

9. How we score a result. ECS asks, for each node, whether its co-members are the same in the ground-truth partition and in the detected cover. 1 is identical and near 0 is unrelated. Left-out nodes count as a community of one, and overlap is allowed.

10. What we did. We rebuilt both variants with tests. E1 checks the paper's setting. E3 adds a rule for the settings. E2 and E4 are backup: an unclean size cut, and real committee data.

11. E1 figure, large-edge communities. Intra-community hyperedges are size 6 to 8 and random hyperedges are size 3 to 5. Dark means good. The large-edge variant is 1.000 at every noise level. The small-edge variant drops from 1.000 to 0.424.

12. E1 figure, the mirror case. Intra-community hyperedges are size 3 to 5 and random hyperedges are size 6 to 8. Now the small-edge variant stays at 1.000 and the large-edge variant drops to 0.206. The right variant stays perfect. The wrong one collapses.

13. Our rule for picking parameters without labels. Try every setting. Throw out settings where one community swallows the rest. Among the others, pick the one with the fewest, largest communities. If nothing survives, use the paper's default. Picking by coverage failed, because tiny fragments give high coverage.

14. E3 figure, rule against oracle and default. The large-edge rule scores 0.555 against a default of 0.292 and an oracle of 0.644. The small-edge rule scores 0.586 against 0.229 and 0.698. It hits the oracle on 46.6 and 55.4 percent of runs.

15. What we found. The right variant is exact when one hyperedge size is pure. With no pure size, both fail together. Clique percolation on the flattened graph is too costly on big hyperedges. Our rule beats the default and trails the oracle.

16. Limits. Planted communities are disjoint. Two party labels are weak. The algorithm leaves nodes out. The rule is a heuristic, and on real committees it gives no gain.

17. Conclusion. The algorithm behaves as the paper says in its setting. It works when one size is pure and degrades when none is. Our rule is a cheap, label-free gain over the default, with a clear gap to the oracle.

18. Backup, E2 explanation. We let intra-community and random hyperedge sizes overlap. With a clean cut or partial overlap, the matching variant still works. With full overlap, both variants fall to 0.601 at noise 0.1 and about 0.24 at 0.8. Clique percolation timed out in 278 of 350 runs.

19. Backup, E2 figure. Read it by layout. The partial-overlap panels stay near 1 for the matching variant. The full-overlap panel collapses for both. The missing clique cells are timeouts.

20. Backup, E4 committee data. House has 1290 nodes and 341 hyperedges, Senate has 282 and 315. Best ECS against party is about 0.5 from one or two communities covering every node. The rule gives no gain, and we state that as a limit.